# Command Line Interface Cheat Sheet

Windows CMD · PowerShell · Bash (Linux and macOS) · Git

**How to read this sheet.** Each row shows the same task in all three shells. Commands are in monospace; *grey italics* explain flags or name an alias for the same command. 🔒 needs administrator, root or sudo. ⚠ destructive; check before running. (Linux) or (macOS) marks a command that exists only on that platform. ★ marks the everyday essentials. Click any command to copy it.

## Everyday Essentials

The commands most people use every day, in short form. Click a task name for the full entry with more options and notes.

| Task | CMD | PowerShell | Bash |
|---|---|---|---|
| ★ **Change directory** | `cd C:\Users`<br>`cd ..\Desktop` | `cd "C:\Program Files"`<br>`Set-Location ~\Desktop` | `cd /home/user/docs`<br>`cd ~/Desktop` |
| ★ **List files** | `dir`<br>`dir /A` *(include hidden)* | `Get-ChildItem` *(alias: ls, dir, gci)*<br>`ls -Force` *(include hidden)* | `ls`<br>`ls -la` *(long, incl. hidden)* |
| ★ **Copy files** | `copy file.txt C:\Backup`<br>`copy *.txt D:\Backup` | `Copy-Item file.txt ~\Backup` *(alias: cp)*<br>`Copy-Item *.txt -Destination C:\Backup` | `cp file.txt ~/Backup`<br>`cp -i file.txt ~/Backup` *(prompt before overwrite)* |
| ★ **Move or rename** | `move file.txt C:\New`<br>`ren old.txt new.txt` | `Move-Item file.txt C:\New` *(alias: mv)*<br>`Rename-Item old.txt new.txt` | `mv file.txt ~/new/`<br>`mv old.txt new.txt` |
| ★ **Delete files** | `del oldfile.txt`<br>`del *.tmp` | `Remove-Item oldfile.txt` *(alias: rm, del)*<br>`Remove-Item *.tmp -WhatIf` *(dry run)* | `rm oldfile.txt`<br>`rm -i file.txt` *(confirm)* |
| ★ **View file contents** | `type readme.txt` | `Get-Content readme.txt` *(alias: cat, type, gc)* | `cat readme.txt`<br>`less readme.txt` *(paginated; q to quit)* |
| ★ **Find files** | `dir /S /B *.txt`<br>`where /R C:\Projects *.log` | `Get-ChildItem -Recurse -Filter *.txt`<br>`gci -r -Include *.log,*.txt` | `find . -name "*.txt"`<br>`find /home -type f -name "*.log"` |
| ★ **Search text in files** | `findstr "error" log.txt`<br>`findstr /S /I "text" *.*` *(recursive, ignore case)* | `Select-String -Path log.txt -Pattern error` *(alias: sls)*<br>`sls error *.log -CaseSensitive` | `grep "error" log.txt`<br>`grep -ri "error" /var/log/` *(recursive, ignore case)* |
| ★ **List processes** | `tasklist`<br>`tasklist /V` *(verbose)* | `Get-Process` *(alias: ps, gps)*<br>`Get-Process -Name chrome` | `ps aux`<br>`ps aux \| grep nginx` |
| ★ **Kill process** | `taskkill /IM notepad.exe`<br>`taskkill /PID 1234 /F` *(force)* | `Stop-Process -Name notepad` *(alias: kill)*<br>`Stop-Process -Id 1234 -Force` | `kill 1234` *(SIGTERM, polite)*<br>`kill -9 1234` *(SIGKILL, force)* |

| Task | CMD | PowerShell | Bash |
|---|---|---|---|
| ★ **Ping host** | `ping google.com` *(4 packets)*<br>`ping -n 10 192.168.1.1` | `Test-Connection google.com`<br>`Test-Connection 8.8.8.8 -Count 5` | `ping google.com` *(until Ctrl+C)*<br>`ping -c 5 google.com` |
| ★ **IP configuration** | `ipconfig`<br>`ipconfig /all` *(MAC, DNS, DHCP)* | `Get-NetIPConfiguration`<br>`Get-NetIPAddress -AddressFamily IPv4` | `ip addr` *(Linux; short form: ip a)*<br>`ifconfig` *(macOS; deprecated on Linux)* |
| ★ **Show environment variables** | `set`<br>`set PATH` *(all starting with PATH)* | `Get-ChildItem Env:` *(or: dir env:)*<br>`$env:PATH` | `printenv`<br>`env` |
| ★ **Command history** | `doskey /history`<br>`F7` *(popup list)* | `Get-History` *(alias: h, history)*<br>`Get-Content (Get-PSReadLineOption).HistorySavePath` *(all sessions)* | `history`<br>`history 20` *(last 20)* |
| ★ **Get help** | `dir /?`<br>`help copy` | `Get-Help Get-Process -Examples`<br>`Get-Help *-Service` *(search by name)* | `man ls`<br>`ls --help` |

## File and Directory Management

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| ★ **Change directory** | `cd C:\Users`<br>`cd ..\Desktop`<br>`cd /d D:\Data` *(switch drive)*<br>`D:` *(switch drive only)* | `cd "C:\Program Files"`<br>`Set-Location ~\Desktop`<br>`cd -` *(previous dir, PS 6+)* | `cd /home/user/docs`<br>`cd ~/Desktop`<br>`cd -` *(previous dir)*<br>`cd` *(home)* | cd .. goes to the parent. ~ is the home directory in PowerShell and Bash. CMD needs /d to change drive and directory at once. |
| ★ **List files** | `dir`<br>`dir /A` *(include hidden)*<br>`dir /S` *(recursive)*<br>`dir /B` *(bare names)*<br>`dir /O-D` *(newest first)* | `Get-ChildItem` *(alias: ls, dir, gci)*<br>`ls -Force` *(include hidden)*<br>`Get-ChildItem -Recurse`<br>`ls *.txt` | `ls`<br>`ls -la` *(long, incl. hidden)*<br>`ls -lh` *(readable sizes)*<br>`ls -lt` *(newest first)*<br>`ls -R` *(recursive)* | |
| **Show current directory** | `cd` *(no arguments)*<br>`echo %CD%` | `Get-Location` *(alias: pwd)*<br>`$PWD` | `pwd` | |
| **Create directory** | `mkdir MyReports`<br>`md Folder1\Folder2` *(nested is fine)* | `mkdir ~/Projects/New`<br>`New-Item -ItemType Directory MyReports` | `mkdir MyReports`<br>`mkdir -p a/b/c` *(nested)* | |
| **Remove directory** | `rmdir OldFolder` *(empty only)*<br>`rmdir /S /Q OldFolder` *(with contents, no prompt)* | `Remove-Item OldFolder -Recurse -Force`<br>`Remove-Item OldFolder -Recurse -WhatIf` *(preview)* | `rmdir OldFolder` *(empty only)*<br>`rm -r OldFolder`<br>`rm -rf OldFolder` *(no prompts)* | ⚠ Recursive deletes bypass the Recycle Bin / Trash. There is no undo. |
| **Directory tree** | `tree C:\Projects`<br>`tree /F` *(include files)* | `tree /F` *(same tree.com as CMD)*<br>`Get-ChildItem -Recurse -Name` | `tree ~/Projects`<br>`tree -L 2` *(limit depth)* | tree is not preinstalled on most Linux distros or macOS: apt install tree, brew install tree. |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| ★ **Copy files** | `copy file.txt C:\Backup`<br>`copy *.txt D:\Backup` | `Copy-Item file.txt ~\Backup` *(alias: cp)*<br>`Copy-Item *.txt -Destination C:\Backup` | `cp file.txt ~/Backup`<br>`cp -i file.txt ~/Backup` *(prompt before overwrite)* | |
| **Copy directories** | `xcopy C:\Src D:\Dst /E /I`<br>`robocopy C:\Src D:\Dst /E`<br>`robocopy C:\Src D:\Dst /MIR` *(mirror; deletes extras)* | `Copy-Item folder -Recurse -Destination dest` | `cp -r folder/ ~/dest`<br>`cp -a folder/ ~/dest` *(preserve attributes)*<br>`rsync -av src/ dest/` | robocopy and rsync are the robust choices for large copies: both can resume and mirror. ⚠ /MIR and rsync --delete remove files at the destination. |
| ★ **Move or rename** | `move file.txt C:\New`<br>`ren old.txt new.txt` | `Move-Item file.txt C:\New` *(alias: mv)*<br>`Rename-Item old.txt new.txt` | `mv file.txt ~/new/`<br>`mv old.txt new.txt` | |
| ★ **Delete files** | `del oldfile.txt`<br>`del *.tmp`<br>`del /P file.txt` *(prompt)* | `Remove-Item oldfile.txt` *(alias: rm, del)*<br>`Remove-Item *.tmp -WhatIf` *(dry run)* | `rm oldfile.txt`<br>`rm -i file.txt` *(confirm)*<br>`rm *.tmp` | ⚠ No Recycle Bin. In PowerShell, -WhatIf previews any destructive cmdlet. |
| ★ **View file contents** | `type readme.txt` | `Get-Content readme.txt` *(alias: cat, type, gc)* | `cat readme.txt`<br>`less readme.txt` *(paginated; q to quit)* | |
| **Create empty file** | `type nul > notes.txt`<br>`copy con notes.txt` *(type text, then Ctrl+Z, Enter)* | `New-Item -ItemType File notes.txt` *(alias: ni)* | `touch notes.txt`<br>`> notes.txt` | touch also updates the timestamp of an existing file. New-Item errors if the file exists unless you add -Force. |
| ★ **Find files** | `dir /S /B *.txt`<br>`where /R C:\Projects *.log` | `Get-ChildItem -Recurse -Filter *.txt`<br>`gci -r -Include *.log,*.txt`<br>`gci -r \| Where LastWriteTime -gt (Get-Date).AddDays(-1)` | `find . -name "*.txt"`<br>`find /home -type f -name "*.log"`<br>`find . -mtime -1` *(modified in last day)*<br>`find . -size +100M` | |
| **Which command runs** | `where python` | `Get-Command python`<br>`Get-Command ls` *(shows what an alias points to)* | `which python`<br>`type ls` *(shows aliases and builtins too)*<br>`command -v python` | |
| **Compare files** | `fc file1.txt file2.txt`<br>`fc /B a.bin b.bin` *(binary)* | `Compare-Object (Get-Content a.txt) (Get-Content b.txt)` *(alias: diff)* | `diff file1.txt file2.txt`<br>`diff -u a.txt b.txt` *(unified format)* | PowerShell's diff alias is not Unix diff: it compares sets of objects and ignores line order unless you add -SyncWindow 0. |
| ★ **Search text in files** | `findstr "error" log.txt`<br>`findstr /S /I "text" *.*` *(recursive, ignore case)*<br>`findstr /N "text" file.txt` *(line numbers)* | `Select-String -Path log.txt -Pattern error` *(alias: sls)*<br>`sls error *.log -CaseSensitive`<br>`gci -r *.log \| sls error` | `grep "error" log.txt`<br>`grep -ri "error" /var/log/` *(recursive, ignore case)*<br>`grep -n "text" file.txt` *(line numbers)* | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **File permissions** | `icacls file.txt`<br>`icacls file.txt /grant User:F` | `Get-Acl file.txt`<br>`Set-Acl file.txt -AclObject $acl`<br>`icacls file.txt` *(works here too)* | `chmod 755 script.sh`<br>`chmod +x script.sh`<br>`chown user:group file.txt` | 🔒 Changing permissions on files you do not own needs admin or sudo. Windows uses ACLs; Unix uses rwx bits. |
| **File attributes** | `attrib +r file.txt` *(read-only)*<br>`attrib +h file.txt` *(hidden)*<br>`attrib -r -h file.txt` | `Set-ItemProperty file.txt IsReadOnly $true`<br>`(Get-Item file.txt).Attributes` | `chattr +i file.txt` *(immutable, Linux)*<br>`chattr -i file.txt`<br>`chflags hidden file.txt` *(macOS)* | Unix has no hidden attribute: files whose names start with a dot are hidden by convention. |
| **Symbolic links** | `mklink link.txt C:\target.txt`<br>`mklink /D linkdir C:\target` | `New-Item -ItemType SymbolicLink -Path link -Target target` | `ln -s target link`<br>`ln -s /path/to/dir linkdir` | 🔒 Windows needs an admin prompt or Developer Mode to create symlinks. Bash argument order is target first, then link. |
| **Compress / archive** | `tar -a -cf archive.zip folder` *(Windows 10 1803+)*<br>`compact /C file.txt` *(NTFS compression, not an archive)* | `Compress-Archive -Path folder -DestinationPath archive.zip` | `tar -czvf archive.tar.gz folder/`<br>`zip -r archive.zip folder/`<br>`gzip file.txt` *(replaces file with file.txt.gz)* | bsdtar ships with Windows 10 and later and works in both CMD and PowerShell. It reads .zip, .tar.gz, .7z and more. |
| **Extract** | `tar -xf archive.zip`<br>`tar -xf archive.tar.gz`<br>`expand archive.cab C:\dest` *(CAB only)* | `Expand-Archive archive.zip -DestinationPath dest\` | `unzip archive.zip`<br>`tar -xzvf archive.tar.gz`<br>`gunzip file.txt.gz` | |
| **File hash / checksum** | `certutil -hashfile file.iso SHA256` | `Get-FileHash file.iso` *(SHA256 by default)*<br>`Get-FileHash file.iso -Algorithm MD5` | `sha256sum file.iso`<br>`md5sum file.iso`<br>`shasum -a 256 file.iso` *(macOS)* | |
| **Open with default app** | `start report.pdf`<br>`start .` *(Explorer here)*<br>`start https://example.com` | `Invoke-Item report.pdf` *(alias: ii)*<br>`start .` *(alias of Start-Process)* | `xdg-open report.pdf` *(Linux)*<br>`open report.pdf` *(macOS)*<br>`open .` *(Finder here, macOS)* | |
| **Clear screen** | `cls` | `Clear-Host` *(alias: cls, clear)*<br>`Ctrl+L` | `clear`<br>`Ctrl+L` | |

## Disk and Storage

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Free disk space** | `fsutil volume diskfree C:`<br>`wmic logicaldisk get name,size,freespace` *(wmic is deprecated)* | `Get-PSDrive -PSProvider FileSystem`<br>`Get-Volume`<br>`Get-CimInstance Win32_LogicalDisk \| Select DeviceID,Size,FreeSpace` | `df -h`<br>`df -h /home` *(one filesystem)* | wmic is deprecated and removed from recent Windows 11 builds. Prefer Get-CimInstance. |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Disk and partition info** | `diskpart` *(interactive: list disk, list volume)*<br>`wmic diskdrive get model,size` *(deprecated)* | `Get-Disk`<br>`Get-Partition`<br>`Get-Volume` | `lsblk` *(Linux)*<br>`sudo fdisk -l` *(Linux)*<br>`diskutil list` *(macOS)* | |
| **Folder size** | `dir /S C:\folder` *(total at the end)* | `(Get-ChildItem -Recurse folder \| Measure-Object Length -Sum).Sum / 1MB` | `du -sh folder/`<br>`du -h --max-depth=1 . \| sort -h` *(Linux)*<br>`du -sh * \| sort -h` | |
| **Check disk** | `chkdsk C:`<br>`chkdsk C: /F` *(fix errors; system drive needs a reboot)* | `Repair-Volume -DriveLetter C -Scan`<br>`Repair-Volume -DriveLetter C -OfflineScanAndFix` | `sudo fsck /dev/sda1` *(unmount first)*<br>`diskutil verifyVolume /` *(macOS)* | 🔒 Requires admin or sudo. Never run fsck on a mounted filesystem. |
| **Format drive** | `format D: /FS:NTFS /Q` | `Format-Volume -DriveLetter D -FileSystem NTFS` | `sudo mkfs.ext4 /dev/sdb1`<br>`diskutil eraseDisk APFS Name disk2` *(macOS)* | ⚠⚠ Destroys everything on the volume. Double-check the drive letter or device name first. |
| **Mount / map drives** | `net use Z: \\server\share` *(map network drive)*<br>`net use Z: /delete`<br>`mountvol` *(list volume mount points)* | `New-PSDrive -Name Z -PSProvider FileSystem -Root \\server\share -Persist`<br>`Mount-DiskImage file.iso`<br>`Dismount-DiskImage file.iso` | `sudo mount /dev/sdb1 /mnt/usb`<br>`sudo umount /mnt/usb`<br>`mount` *(list mounts)* | |

## Windows Repair

| Task | Command | Notes |
|---|---|---|
| **Repair system files (SFC)** | `sfc /scannow` *(scan and repair protected system files)*<br>`sfc /verifyonly` *(scan only, no repair)*<br>`sfc /scannow /offbootdir=C:\ /offwindir=C:\Windows` *(offline, from recovery)* | 🔒 Run from an elevated prompt. If SFC reports files it could not fix, run the DISM RestoreHealth step below, then run sfc /scannow again. |
| **Repair the component store (DISM)** | `DISM /Online /Cleanup-Image /CheckHealth` *(quick flag check)*<br>`DISM /Online /Cleanup-Image /ScanHealth` *(full scan, no repair)*<br>`DISM /Online /Cleanup-Image /RestoreHealth` *(download and repair)* | 🔒 Elevated. RestoreHealth fixes the store SFC copies from, so run it first when files are corrupt. It uses Windows Update; add /Source:wim:X:\sources\install.wim:1 /LimitAccess to repair from an install image offline. |
| **Check disk (CHKDSK)** | `chkdsk C:` *(read-only report)*<br>`chkdsk C: /F` *(fix file-system errors)*<br>`chkdsk C: /R` *(find bad sectors, implies /F)* | 🔒 Elevated. The system drive cannot be locked while Windows is running, so the check is scheduled for the next reboot. Also listed under Disk and Storage. |
| **System Restore** | `rstrui` *(open the System Restore wizard)*<br>`Get-ComputerRestorePoint` *(PowerShell: list restore points)*<br>`Checkpoint-Computer -Description "Before change"` *(PowerShell: create one)* | 🔒 Creating or restoring needs admin. System Protection must be turned on for the drive first (System Properties > System Protection). |

| Task | Command | Notes |
|---|---|---|
| **Boot to recovery options** | `shutdown /r /o /t 0` *(reboot into Advanced Startup)*<br>`reagentc /info` *(show recovery environment status)* | Advanced Startup leads to Startup Repair, Safe Mode, System Restore and a command prompt. Or hold Shift while clicking Restart. |

## System and User Information

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Current user** | `whoami`<br>`echo %USERNAME%`<br>`whoami /groups` | `whoami`<br>`$env:USERNAME`<br>`[Environment]::UserName` | `whoami`<br>`echo $USER`<br>`id` *(uid, gid and groups)* | |
| **Hostname** | `hostname`<br>`echo %COMPUTERNAME%` | `hostname`<br>`$env:COMPUTERNAME` | `hostname`<br>`hostnamectl` *(Linux with systemd)*<br>`scutil --get ComputerName` *(macOS)* | |
| **System information** | `systeminfo`<br>`systeminfo \| findstr /B /C:"OS Name" /C:"OS Version"` | `Get-ComputerInfo`<br>`Get-ComputerInfo \| Select OsName,OsVersion,CsTotalPhysicalMemory`<br>`Get-CimInstance Win32_OperatingSystem` | `uname -a`<br>`cat /etc/os-release` *(Linux)*<br>`hostnamectl` *(Linux)*<br>`sw_vers` *(macOS)* | Get-ComputerInfo is slow (several seconds). Pick properties with Select-Object. |
| **OS version** | `ver` | `[Environment]::OSVersion`<br>`$PSVersionTable.PSVersion` *(PowerShell version)* | `uname -r` *(kernel)*<br>`lsb_release -a` *(Debian/Ubuntu)*<br>`sw_vers -productVersion` *(macOS)* | |
| **CPU** | `echo %NUMBER_OF_PROCESSORS%`<br>`wmic cpu get name,numberofcores` *(deprecated)* | `Get-CimInstance Win32_Processor \| Select Name,NumberOfCores,NumberOfLogicalProcessors` | `lscpu` *(Linux)*<br>`nproc` *(Linux)*<br>`sysctl -n machdep.cpu.brand_string` *(macOS)* | |
| **Memory** | `systeminfo \| findstr /C:"Total Physical Memory" /C:"Available Physical Memory"` | `Get-CimInstance Win32_OperatingSystem \| Select TotalVisibleMemorySize,FreePhysicalMemory`<br>`Get-CimInstance Win32_PhysicalMemory \| Select Capacity,Speed` | `free -h` *(Linux)*<br>`cat /proc/meminfo` *(Linux)*<br>`vm_stat` *(macOS)*<br>`top` *(any)* | |
| **Uptime** | `systeminfo \| findstr /C:"System Boot Time"`<br>`net statistics workstation \| findstr "since"` | `(Get-Date) - (Get-CimInstance Win32_OperatingSystem).LastBootUpTime`<br>`Get-Uptime` *(PowerShell 6+ only)* | `uptime`<br>`uptime -p` *(pretty, Linux)*<br>`uptime -s` *(boot time, Linux)* | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Date and time** | `date /T`<br>`time /T`<br>`echo %DATE% %TIME%` | `Get-Date`<br>`Get-Date -Format "yyyy-MM-dd HH:mm:ss"`<br>`Get-Date -UFormat "%Y-%m-%d"` | `date`<br>`date +"%Y-%m-%d %H:%M:%S"`<br>`date -u` *(UTC)* | Without /T, CMD's date and time commands prompt you to change the clock (🔒 admin). |
| **Logged-in users** | `query user` *(alias: quser)* | `query user`<br>`Get-CimInstance Win32_ComputerSystem \| Select UserName` | `who`<br>`w` *(who, plus what they are running)*<br>`last` *(login history)* | |

## Processes, Services and Power

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| ★ **List processes** | `tasklist`<br>`tasklist /V` *(verbose)*<br>`tasklist /FI "IMAGENAME eq chrome.exe"` | `Get-Process` *(alias: ps, gps)*<br>`Get-Process -Name chrome`<br>`Get-Process \| Sort-Object CPU -Descending \| Select -First 10` | `ps aux`<br>`ps aux \| grep nginx`<br>`top` *(live)*<br>`htop` *(live, nicer; needs install)* | |
| ★ **Kill process** | `taskkill /IM notepad.exe`<br>`taskkill /PID 1234 /F` *(force)*<br>`taskkill /F /IM chrome.exe /T` *(with child processes)* | `Stop-Process -Name notepad` *(alias: kill)*<br>`Stop-Process -Id 1234 -Force` | `kill 1234` *(SIGTERM, polite)*<br>`kill -9 1234` *(SIGKILL, force)*<br>`pkill -f pattern`<br>`killall firefox` | ⚠ Force-killing skips cleanup and can corrupt files being written. Try the polite form first. |
| **Find process using a port** | `netstat -ano \| findstr :8080` *(PID in last column)* | `Get-NetTCPConnection -LocalPort 8080 \| Select OwningProcess`<br>`Get-Process -Id (Get-NetTCPConnection -LocalPort 8080).OwningProcess` | `sudo lsof -i :8080`<br>`sudo ss -ltnp \| grep 8080` *(Linux)* | |
| **Run in background** | `start /B program.exe` *(same window)*<br>`start "" notepad.exe` *(new window)* | `Start-Process notepad`<br>`Start-Job { long-task }; Get-Job; Receive-Job 1`<br>`command &` *(PowerShell 6+)* | `command &`<br>`nohup command &` *(survives logout)*<br>`jobs / fg / bg`<br>`Ctrl+Z` *(suspend the foreground job)* | |
| **List services** | `sc query` *(running)*<br>`sc query state= all`<br>`net start` *(running only)* | `Get-Service`<br>`Get-Service \| Where Status -eq Running`<br>`Get-Service -Name Spooler` | `systemctl list-units --type=service` *(Linux)*<br>`systemctl list-units --type=service --state=running`<br>`launchctl list` *(macOS)* | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Start / stop / restart service** | `sc start Spooler`<br>`sc stop Spooler`<br>`net stop Spooler && net start Spooler`<br>*(net start/stop also work)* | `Start-Service Spooler`<br>`Stop-Service Spooler`<br>`Restart-Service Spooler` | `sudo systemctl start nginx`<br>`sudo systemctl stop nginx`<br>`sudo systemctl restart nginx`<br>`sudo systemctl enable nginx` *(start at boot)* | 🔒 Requires admin or sudo. sc and Get-Service use the service name (Spooler), not the display name (Print Spooler). |
| **Service status** | `sc query Spooler`<br>`sc qc Spooler` *(configuration)* | `Get-Service Spooler`<br>`(Get-Service Spooler).Status` | `systemctl status nginx`<br>`systemctl is-active nginx`<br>`journalctl -u nginx -f` *(follow logs, Linux)* | |
| **Scheduled tasks** | `schtasks /query`<br>`schtasks /create /tn Backup /tr C:\backup.bat /sc daily /st 02:00`<br>`schtasks /delete /tn Backup` | `Get-ScheduledTask`<br>`Get-ScheduledTask Backup \| Start-ScheduledTask`<br>`Register-ScheduledTask` *(see the Get-Help examples)* | `crontab -l` *(list)*<br>`crontab -e` *(edit)*<br>`0 2 * * * /path/backup.sh` *(daily at 02:00)*<br>`launchctl` *(macOS launchd)* | |
| **Restart system** | `shutdown /r /t 0`<br>`shutdown /a` *(abort a pending shutdown)* | `Restart-Computer`<br>`Restart-Computer -Force` | `sudo reboot`<br>`sudo shutdown -r now`<br>`sudo shutdown -r +5` *(in 5 minutes)* | 🔒 Requires admin or sudo. |
| **Shut down system** | `shutdown /s /t 0`<br>`shutdown /s /t 60` *(in 60 seconds)*<br>`shutdown /h` *(hibernate)* | `Stop-Computer`<br>`Stop-Computer -Force` | `sudo shutdown -h now`<br>`sudo poweroff`<br>`sudo shutdown -c` *(cancel, Linux)* | 🔒 Requires admin or sudo. |

## Networking and Diagnostics

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| ★ **Ping host** | `ping google.com` *(4 packets)*<br>`ping -n 10 192.168.1.1`<br>`ping -t host` *(until Ctrl+C)* | `Test-Connection google.com`<br>`Test-Connection 8.8.8.8 -Count 5`<br>`Test-Connection host -Quiet` *(True/False)* | `ping google.com` *(until Ctrl+C)*<br>`ping -c 5 google.com` | Windows ping stops after 4 packets by default. Unix ping runs until you stop it. |
| ★ **IP configuration** | `ipconfig`<br>`ipconfig /all` *(MAC, DNS, DHCP)*<br>`ipconfig /release && ipconfig /renew` | `Get-NetIPConfiguration`<br>`Get-NetIPAddress -AddressFamily IPv4` | `ip addr` *(Linux; short form: ip a)*<br>`ifconfig` *(macOS; deprecated on Linux)*<br>`hostname -I` *(Linux, addresses only)* | |
| **Public IP** | `curl ifconfig.me` | `Invoke-RestMethod ifconfig.me` | `curl ifconfig.me` | |
| **Connections and open ports** | `netstat -an` *(all)*<br>`netstat -ano` *(with PIDs)*<br>`netstat -b` *(with program names, 🔒 admin)* | `Get-NetTCPConnection`<br>`Get-NetTCPConnection -State Listen` | `ss -tulpn` *(Linux, preferred)*<br>`sudo netstat -tulpn` *(older Linux)*<br>`sudo lsof -i -P` *(macOS and Linux)* | netstat and ifconfig are deprecated on Linux in favour of ss and ip, but still work on macOS. |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **DNS lookup** | `nslookup google.com`<br>`nslookup google.com 8.8.8.8` *(ask a specific server)*<br>`nslookup -type=MX google.com` | `Resolve-DnsName google.com`<br>`Resolve-DnsName google.com -Type MX`<br>`Resolve-DnsName google.com -Server 8.8.8.8` | `dig google.com`<br>`dig +short google.com`<br>`dig MX google.com`<br>`host google.com` | |
| **Trace route** | `tracert google.com`<br>`tracert -d google.com` *(skip DNS, faster)*<br>`pathping google.com` | `Test-NetConnection google.com -TraceRoute` | `traceroute google.com`<br>`tracepath google.com` *(Linux, no root needed)*<br>`mtr google.com` *(live; needs install)* | |
| **Routing table** | `route print`<br>`route add 192.168.2.0 mask 255.255.255.0 192.168.1.1` | `Get-NetRoute`<br>`New-NetRoute -DestinationPrefix 192.168.2.0/24 -NextHop 192.168.1.1 -InterfaceIndex 12` | `ip route` *(Linux)*<br>`sudo ip route add 192.168.2.0/24 via 192.168.1.1`<br>`netstat -rn` *(macOS)* | 🔒 Adding routes needs admin or sudo. |
| **Flush DNS cache** | `ipconfig /flushdns`<br>`ipconfig /displaydns` | `Clear-DnsClientCache`<br>`Get-DnsClientCache` | `sudo resolvectl flush-caches` *(Linux, systemd-resolved)*<br>`sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder` *(macOS)* | |
| **ARP table** | `arp -a` | `Get-NetNeighbor`<br>`Get-NetNeighbor -State Reachable` | `ip neigh` *(Linux)*<br>`arp -a` *(macOS and Linux)* | |
| **Download file** | `curl -O https://example.com/file.zip`<br>`curl -L -o out.zip URL` *(follow redirects, custom name)*<br>`certutil -urlcache -split -f URL file.zip` *(legacy)* | `Invoke-WebRequest URL -OutFile file.zip` *(alias: iwr)*<br>`Invoke-RestMethod URL (parses JSON)` *(alias: irm)*<br>`curl.exe -O URL` | `wget URL`<br>`curl -O URL`<br>`curl -L -o out.zip URL` | In Windows PowerShell 5.1, curl is an alias of Invoke-WebRequest. Type curl.exe to get the real tool. |
| **Test a port** | `curl -v telnet://host:443`<br>`telnet host 80` *(optional feature, off by default)* | `Test-NetConnection host -Port 443` *(alias: tnc)*<br>`tnc host -Port 443 -InformationLevel Quiet` | `nc -zv host 443`<br>`curl -v telnet://host:443`<br>`(echo > /dev/tcp/host/443) && echo open` *(Bash only)* | |
| **SSH and remote copy** | `ssh user@host`<br>`scp file.txt user@host:/path/`<br>`ssh -i key.pem user@host` | `ssh user@host`<br>`scp file.txt user@host:/path/`<br>`Enter-PSSession -ComputerName host` *(WinRM, Windows to Windows)* | `ssh user@host`<br>`scp file.txt user@host:/path/`<br>`rsync -avz src/ user@host:/dest/` | The OpenSSH client is built into Windows 10 1809 and later. |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Wi-Fi profiles and passwords** | `netsh wlan show profiles` *(saved networks)*<br>`netsh wlan show profile name="MyWiFi" key=clear` *(reveals the password)*<br>`netsh wlan connect name="MyWiFi"` | `netsh wlan show profiles`<br>`netsh wlan show profile name="MyWiFi" key=clear \| sls Key` *(just the password line)* | `nmcli device wifi list` *(Linux)*<br>`sudo nmcli -s -g 802-11-wireless-security.psk connection show "MyWiFi"` *(Linux password)*<br>`security find-generic-password -wa "MyWiFi"` *(macOS; prompts for keychain)* | The Windows password shows as "Key Content" in the output. On Linux the psk read needs sudo; macOS prompts for your login keychain. |
| **Firewall** | `netsh advfirewall show allprofiles`<br>`netsh advfirewall firewall add rule name="App" dir=in action=allow program="C:\app.exe" enable=yes`<br>`netsh advfirewall set allprofiles state on` | `Get-NetFirewallProfile`<br>`New-NetFirewallRule -DisplayName "App" -Direction Inbound -Action Allow -Program "C:\app.exe"`<br>`Set-NetFirewallProfile -All -Enabled True` | `sudo ufw status`<br>`sudo ufw allow 22/tcp` *(Debian/Ubuntu)*<br>`sudo firewall-cmd --add-port=8080/tcp --permanent` *(Fedora/RHEL)*<br>`sudo pfctl -sr` *(macOS rules)* | 🔒 Requires admin or sudo. ufw is the friendly front end on Ubuntu; firewalld (firewall-cmd) on Fedora/RHEL; macOS uses pf plus the Application Firewall in System Settings. |

## Environment, History and Shell Basics

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| ★ **Show environment variables** | `set`<br>`set PATH` *(all starting with PATH)*<br>`echo %PATH%` | `Get-ChildItem Env:` *(or: dir env:)*<br>`$env:PATH`<br>`$env:PATH -split ';'` *(one per line)* | `printenv`<br>`env`<br>`echo $PATH`<br>`echo $PATH \| tr ':' '\n'` *(one per line)* | |
| **Set environment variable** | `set MYVAR=value` *(this session)*<br>`setx MYVAR value` *(permanent, new windows only)*<br>`setx PATH "%PATH%;C:\tools"` *(⚠ truncates at 1024 chars)* | `$env:MYVAR = 'value'` *(this session)*<br>`[Environment]::SetEnvironmentVariable('MYVAR','value','User')` *(permanent)* | `export MYVAR=value` *(this session)*<br>`MYVAR=value command` *(one command only)*<br>`add the export line to ~/.bashrc or ~/.zshrc` *(permanent)* | Permanent changes only affect shells opened afterwards. |
| ★ **Command history** | `doskey /history`<br>`F7` *(popup list)* | `Get-History` *(alias: h, history)*<br>`Get-Content (Get-PSReadLineOption).HistorySavePath` *(all sessions)* | `history`<br>`history 20` *(last 20)*<br>`history \| grep ssh` | |
| **Re-run a command** | `F3 or ↑` *(previous)* | `↑` *(previous)*<br>`Invoke-History 5` *(alias: r 5)* | `!!` *(previous)*<br>`!5` *(history entry 5)*<br>`!ssh` *(last command starting with ssh)*<br>`sudo !!` *(repeat with sudo)* | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Search history** | `F7` *(dialog)*<br>`type a prefix, then F8` *(cycle matches)* | `Ctrl+R` *(reverse search)*<br>`type a prefix, then F8`<br>`Get-History \| Where CommandLine -like '*git*'` | `Ctrl+R` *(reverse search)*<br>`history \| grep pattern` | |
| **Aliases** | `doskey ll=dir /B $*` *(this session)*<br>*(persist via a startup script in the AutoRun registry key)* | `Set-Alias ll Get-ChildItem`<br>`Get-Alias` *(list all)*<br>`function gs { git status }` *(aliases cannot take arguments; use a function)* | `alias ll='ls -la'`<br>`alias` *(list all)*<br>`unalias ll` | |
| **Shell profile / startup** | `HKCU\Software\Microsoft\Command Processor\AutoRun` *(registry value)* | `$PROFILE` *(path to your profile script)*<br>`notepad $PROFILE`<br>`. $PROFILE` *(reload)* | `~/.bashrc` *(interactive Bash)*<br>`~/.bash_profile` *(login shells)*<br>`~/.zshrc` *(zsh; default on macOS)*<br>`source ~/.bashrc` *(reload)* | Put aliases, functions and PATH changes in the profile so they survive between sessions. |
| **Exit code of last command** | `echo %ERRORLEVEL%`<br>`if %ERRORLEVEL% neq 0 echo failed` | `$?` *(True or False)*<br>`$LASTEXITCODE` *(from the last native .exe)* | `echo $?` *(0 means success)* | |
| **Chain commands** | `a && b` *(b only if a succeeds)*<br>`a \|\| b` *(b only if a fails)*<br>`a & b` *(both, regardless)* | `a; b` *(both, regardless)*<br>`a && b, a \|\| b` *(PowerShell 7+ only)*<br>*(-and / -or are boolean operators, not chaining)* | `a && b`<br>`a \|\| b`<br>`a; b` | |
| **★ Get help** | `dir /?`<br>`help copy` | `Get-Help Get-Process -Examples`<br>`Get-Help *-Service` *(search by name)*<br>`Get-Command -Noun Service`<br>`Update-Help` *(once, 🔒 admin)* | `man ls`<br>`ls --help`<br>`tldr ls` *(needs install)* | |
| **Run as admin / root** | `runas /user:Administrator cmd`<br>*(or right-click the shell and choose Run as administrator)* | `Start-Process powershell -Verb RunAs`<br>`sudo command` *(Windows 11 24H2+; enable in Settings > System > For developers)* | `sudo command`<br>`sudo -i` *(root shell)*<br>`su -` *(switch user)* | |

## Text Processing and Output

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Display text** | `echo Hello World`<br>`echo %PATH%`<br>`echo.` *(blank line)* | `Write-Output 'Hello'` *(alias: echo, write)*<br>`Write-Host 'Text' -ForegroundColor Red` *(console only, not pipeable)*<br>`"Value: $var"` *(double quotes expand variables)* | `echo "Hello World"`<br>`printf "%s\n" "text"`<br>`echo -e "a\tb"` *(interpret escapes)* | |
| **View a page at a time** | `more file.txt`<br>`dir /S \| more` | `Get-Content file.txt \| more`<br>`Get-Content file.txt \| Out-Host -Paging` | `less file.txt` *(q quits, / searches)*<br>`more file.txt`<br>`cat file.txt \| less` | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **First lines of a file** | `more +5 file.txt` *(skip the first 5 instead)*<br>*(none; use PowerShell)* | `Get-Content file.txt -Head 10`<br>`Get-Content file.txt \| Select-Object -First 10` | `head file.txt` *(10 lines)*<br>`head -n 20 file.txt`<br>`head -c 100 file.bin` *(bytes)* | |
| **Last lines of a file** | *(none; use PowerShell)* | `Get-Content file.txt -Tail 10`<br>`Get-Content log.txt -Wait` *(follow, like tail -f)* | `tail file.txt`<br>`tail -n 20 file.txt`<br>`tail -f log.txt` *(follow)* | |
| **Count lines / words** | `find /c /v "" < file.txt` *(lines)*<br>`type file.txt \| find /c "error"` *(matching lines)* | `Get-Content file.txt \| Measure-Object -Line -Word -Character`<br>`(Get-Content file.txt).Count` *(lines)* | `wc -l file.txt` *(lines)*<br>`wc -w file.txt` *(words)*<br>`grep -c error file.txt` *(matching lines)* | |
| **Sort lines** | `sort file.txt`<br>`sort /R file.txt` *(reverse)*<br>`dir /B \| sort` | `Get-Content file.txt \| Sort-Object`<br>`Sort-Object -Unique`<br>`Get-Process \| Sort-Object CPU -Descending` | `sort file.txt`<br>`sort -r (reverse), -n (numeric), -u` *(unique)*<br>`sort -k2 file.txt` *(by second field)* | |
| **Unique lines** | *(none; use PowerShell)* | `Get-Content file.txt \| Sort-Object -Unique`<br>`Get-Content file.txt \| Group-Object \| Sort Count -Desc` *(with counts)* | `sort -u file.txt`<br>`sort file.txt \| uniq -c` *(with counts)*<br>`uniq file.txt` *(adjacent duplicates only)* | |
| **Filter a stream** | `dir \| find "txt"`<br>`tasklist \| findstr /I chrome` | `Get-Process \| Where-Object Name -like 'chrome*'` *(alias: where, ?)*<br>`Get-Process \| Where CPU -gt 100`<br>`gc log.txt \| sls error` | `ps aux \| grep chrome`<br>`grep -v debug log.txt` *(invert match)* | PowerShell pipes objects, not text: filter on properties with Where-Object, then shape output with Select-Object or Format-Table. |
| **Find and replace** | *(none; use PowerShell)* | `(Get-Content file.txt) -replace 'old','new' \| Set-Content file.txt`<br>`'text' -replace '(\d+)','[$1]'` *(regex by default)* | `sed 's/old/new/g' file.txt` *(prints result)*<br>`sed -i 's/old/new/g' file.txt` *(in place, Linux)*<br>`sed -i '' 's/old/new/g' file.txt` *(in place, macOS)* | The parentheses around Get-Content read the whole file first so PowerShell can safely overwrite it. |
| **Extract columns / fields** | `for /f "tokens=2 delims=," %i in (file.csv) do @echo %i`<br>*(inside a .bat file use %%i)* | `Import-Csv data.csv \| Select-Object Name,Email`<br>`'a,b,c' -split ','`<br>`Get-Content f.txt \| ForEach { ($_ -split '\s+')[1] }` | `cut -d, -f2 file.csv`<br>`awk '{print $2}' file.txt`<br>`awk -F: '{print $1}' /etc/passwd` | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Shape and export output** | `dir /B /O-D` *(bare, newest first)*<br>`tasklist /FO CSV`<br>`tasklist /FO CSV > procs.csv` | `Get-Process \| Select-Object Name,CPU`<br>`Get-Process \| Format-Table -AutoSize`<br>`Get-Process \| Format-List *`<br>`Get-Process \| Export-Csv procs.csv -NoTypeInformation`<br>`Get-Process \| ConvertTo-Json`<br>`Get-Service \| Out-GridView` | `column -t file.txt` *(align columns)*<br>`ls -l \| awk '{print $9, $5}'`<br>`jq . file.json` *(needs install)* | Format-* cmdlets must be last in a pipeline; their output is for the screen only. |
| **Clipboard** | `dir \| clip` | `Get-Process \| Set-Clipboard` *(alias: scb)*<br>`Get-Clipboard` *(alias: gcb)*<br>`ls \| clip` | `ls \| pbcopy (macOS), pbpaste`<br>`ls \| xclip -selection clipboard` *(Linux, X11)*<br>`ls \| wl-copy` *(Linux, Wayland)* | |

## Redirection and Piping

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Redirect output (overwrite) >** | `dir > list.txt` | `Get-Process > processes.txt`<br>`Get-Process \| Out-File -Encoding utf8 p.txt`<br>`Get-Process \| Set-Content p.txt` | `ls -la > listing.txt` | Windows PowerShell 5.1 writes UTF-16 with >. Use Out-File -Encoding utf8 for files other tools will read. PowerShell 7 defaults to UTF-8. |
| **Redirect output (append) >>** | `echo text >> file.txt` | `Get-Date >> log.txt`<br>`Add-Content log.txt 'line'` | `echo "new line" >> file.txt` | |
| **Redirect input <** | `sort < unsorted.txt` | `Get-Content input.txt \| Sort-Object`<br>*(PowerShell has no < operator)* | `sort < unsorted.txt`<br>`cat <<EOF ... EOF` *(here-document)*<br>`cmd <<< "string"` *(here-string)* | |
| **Pipe \|** | `dir \| find "txt"`<br>`type file.txt \| more` | `Get-Process \| Sort-Object CPU -Descending \| Select -First 5`<br>`Get-ChildItem \| Where Length -gt 1MB` | `ls -la \| grep txt`<br>`cat file.txt \| sort \| uniq -c` | CMD and Bash pipe text. PowerShell pipes objects; $_ (or $PSItem) is the current object inside a script block. |
| **Redirect errors 2>** | `command 2> errors.txt`<br>`command 2>&1` *(merge errors into output)* | `command 2> errors.txt`<br>`command 2>&1`<br>`$ErrorActionPreference = 'Stop'` *(make errors terminate)* | `command 2> errors.txt`<br>`command 2>&1` | |
| **Redirect everything** | `command > out.txt 2>&1` | `command *> all.txt`<br>`command 3>&1 (warnings), 4>&1 (verbose), 6>&1` *(information)* | `command &> out.txt`<br>`command > out.txt 2>&1` | |

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Suppress output** | `command > nul`<br>`command > nul 2>&1` | `command \| Out-Null`<br>`command > $null`<br>`$null = command` *(fastest)* | `command > /dev/null`<br>`command &> /dev/null`<br>`command 2> /dev/null` *(errors only)* | |
| **Tee (file and screen)** | *(none; use PowerShell)* | `Get-Process \| Tee-Object -FilePath p.txt`<br>`Get-Process \| Tee-Object -Variable procs \| Select Name` | `ls \| tee listing.txt`<br>`ls \| tee -a listing.txt` *(append)*<br>`echo x \| sudo tee /etc/file` *(write as root)* | |
| **Command substitution** | `for /f %i in ('date /T') do set today=%i` | `$today = Get-Date -Format yyyy-MM-dd`<br>`Write-Output "Today: $(Get-Date)"`<br>`Copy-Item a.txt "backup-$(Get-Date -f yyyyMMdd).txt"` | `today=$(date +%F)`<br>`echo "Today: $(date)"`<br>`cp a.txt "backup-$(date +%F).txt"` | |

## Users and Groups

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **List users** | `net user`<br>`net user alice` *(details)* | `Get-LocalUser`<br>`Get-LocalUser \| Where Enabled` | `cat /etc/passwd` *(Linux)*<br>`getent passwd`<br>`dscl . list /Users` *(macOS)* | |
| **Add user** | `net user alice * /add` *(prompts for password)* | `New-LocalUser alice -Password (Read-Host -AsSecureString)`<br>`New-LocalUser alice -NoPassword` | `sudo adduser alice` *(interactive, Debian/Ubuntu)*<br>`sudo useradd -m alice && sudo passwd alice` | 🔒 Requires admin or sudo. Avoid typing passwords on the command line: they end up in history. |
| **Delete user** | `net user alice /delete` | `Remove-LocalUser alice` | `sudo userdel alice`<br>`sudo userdel -r alice` *(also remove home dir)* | 🔒 Requires admin or sudo. |
| **Change password** | `net user alice *` *(prompts)* | `Set-LocalUser alice -Password (Read-Host -AsSecureString)` | `passwd` *(your own)*<br>`sudo passwd alice` | |
| **List groups** | `net localgroup`<br>`net localgroup Administrators` *(members)* | `Get-LocalGroup`<br>`Get-LocalGroupMember Administrators` | `groups` *(current user)*<br>`getent group`<br>`id alice` | |
| **Add user to group** | `net localgroup Administrators alice /add` | `Add-LocalGroupMember -Group Administrators -Member alice` | `sudo usermod -aG sudo alice` *(Debian/Ubuntu)*<br>`sudo usermod -aG wheel alice` *(Fedora/RHEL)*<br>*(log out and back in to apply)* | 🔒 Requires admin or sudo. ⚠ Forgetting -a in usermod removes the user from all other groups. |

## Package Management

| Task | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **List installed** | `winget list` | `winget list`<br>`choco list` *(Chocolatey)*<br>`Get-Module -ListAvailable` *(PowerShell modules)* | `apt list --installed` *(Debian/Ubuntu)*<br>`dnf list installed` *(Fedora/RHEL)*<br>`brew list` *(macOS)*<br>`pacman -Q` *(Arch)* | wmic product get name is deprecated and very slow; use winget instead. |
| **Search** | `winget search vscode` | `winget search vscode`<br>`choco search vscode`<br>`Find-Module PSReadLine` | `apt search vscode`<br>`dnf search vscode`<br>`brew search vscode` | |
| **Show package info** | `winget show Git.Git` | `winget show Git.Git` | `apt show git`<br>`dnf info git`<br>`brew info git` | |
| **Install** | `winget install Git.Git`<br>`winget install --id Git.Git -e` *(exact id)* | `winget install Git.Git`<br>`choco install git -y`<br>`Install-Module PSReadLine` *(PowerShell module)* | `sudo apt install git`<br>`sudo dnf install git`<br>`brew install git`<br>`sudo pacman -S git` | 🔒 Usually needs admin or sudo. brew and per-user winget installs are the exception. |
| **Update** | `winget upgrade` *(list available)*<br>`winget upgrade --all` | `winget upgrade --all`<br>`choco upgrade all`<br>`Update-Module` | `sudo apt update && sudo apt upgrade`<br>`sudo dnf upgrade`<br>`brew update && brew upgrade`<br>`sudo pacman -Syu` | apt update only refreshes the package index. apt upgrade does the installing. |
| **Remove** | `winget uninstall Git.Git` | `winget uninstall Git.Git`<br>`choco uninstall git`<br>`Uninstall-Module name` | `sudo apt remove git` *(keeps config)*<br>`sudo apt purge git` *(removes config)*<br>`sudo apt autoremove` *(orphaned dependencies)*<br>`brew uninstall git` | |

# Developer Setup

| Task | Windows | macOS | Linux | Notes |
|---|---|---|---|---|
| **Install WSL (Linux on Windows)** | `wsl --install` *(Ubuntu by default; reboot afterwards)*<br>`wsl --install -d Debian` *(pick a distro)*<br>`wsl -l -v` *(list distros and versions)*<br>`wsl --update`<br>`wsl --shutdown`<br>`wsl` *(open the default distro)* | *(not needed; macOS is already Unix)* | *(not needed)* | 🔒 Run from an administrator prompt. Inside WSL, Windows drives are under /mnt/c and Windows tools are on the PATH, so code . opens VS Code. |
| **Package manager** | `winget --version` *(built in on Windows 10 1709+ and 11)*<br>`winget source update`<br>*(Chocolatey is optional: see chocolatey.org/install)* | `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`<br>`brew doctor` | *(apt, dnf or pacman is already installed)*<br>`sudo apt update` *(refresh the index first)* | If winget is missing, install "App Installer" from the Microsoft Store. |
| **Git** | `winget install --id Git.Git -e`<br>`git --version` | `xcode-select --install` *(Apple's git plus compilers)*<br>`brew install git` *(newer version)* | `sudo apt install git`<br>`sudo dnf install git` | On Windows this also installs Git Bash. |
| **GitHub CLI** | `winget install --id GitHub.cli -e`<br>`gh auth login` | `brew install gh`<br>`gh auth login` | `sudo apt install gh` *(Ubuntu 23.04+; older distros: see cli.github.com)*<br>`gh auth login` | gh auth login signs in through the browser and stores credentials that git push uses too. |
| **SSH key for GitHub** | `ssh-keygen -t ed25519 -C "you@example.com"`<br>`Get-Service ssh-agent \| Set-Service -StartupType Automatic; Start-Service ssh-agent` *(PowerShell, 🔒 admin)*<br>`ssh-add $env:USERPROFILE\.ssh\id_ed25519` *(PowerShell)*<br>`gh ssh-key add ~/.ssh/id_ed25519.pub` | `ssh-keygen -t ed25519 -C "you@example.com"`<br>`ssh-add --apple-use-keychain ~/.ssh/id_ed25519`<br>`gh ssh-key add ~/.ssh/id_ed25519.pub` | `ssh-keygen -t ed25519 -C "you@example.com"`<br>`eval "$(ssh-agent -s)"; ssh-add ~/.ssh/id_ed25519`<br>`gh ssh-key add ~/.ssh/id_ed25519.pub` | Test with ssh -T git@github.com. The Windows ssh-agent service is off by default. |
| **Node.js** | `winget install OpenJS.NodeJS.LTS`<br>`winget install Schniz.fnm` *(version manager)*<br>`node -v; npm -v` | `brew install node`<br>`brew install fnm` *(version manager)* | `curl -fsSL https://fnm.vercel.app/install \| bash` *(version manager)*<br>`fnm install --lts`<br>*(distro packages are often years old)* | A version manager such as fnm or nvm lets you switch Node versions per project. |
| **Python** | `winget install Python.Python.3.12`<br>`python --version`<br>`py -3` *(launcher; picks an installed version)* | `brew install python`<br>`python3 --version` | `sudo apt install python3 python3-pip python3-venv`<br>`python3 --version` | Per-project environment: python -m venv .venv, then .venv\Scripts\activate (Windows) or source .venv/bin/activate. |

| Task | Windows | macOS | Linux | Notes |
|---|---|---|---|---|
| **VS Code** | `winget install Microsoft.VisualStudioCode`<br>`code .` *(open the current folder)* | `brew install --cask visual-studio-code`<br>`code .` *(first run Shell Command: Install code from the Command Palette)* | `sudo snap install code --classic`<br>*(or the .deb / .rpm from code.visualstudio.com)*<br>`code .` | |
| **PowerShell 7 and Windows Terminal** | `winget install Microsoft.PowerShell`<br>`winget install Microsoft.WindowsTerminal`<br>`pwsh` *(start PowerShell 7)* | `brew install --cask powershell`<br>`pwsh` | *(see Microsoft's install page for your distro)*<br>`pwsh` | Windows PowerShell 5.1 is built in. PowerShell 7 (pwsh) is the current cross-platform version and installs alongside it. Windows 11 already includes Windows Terminal. |
| **Compilers and build tools** | `winget install Microsoft.VisualStudio.2022.BuildTools`<br>*(choose "Desktop development with C++" in the installer)* | `xcode-select --install` | `sudo apt install build-essential`<br>`sudo dnf groupinstall "Development Tools"` | Needed by npm and pip packages that compile native code. |
| **Docker** | `winget install Docker.DockerDesktop`<br>`docker run hello-world` | `brew install --cask docker`<br>`docker run hello-world` | `curl -fsSL https://get.docker.com \| sh`<br>`sudo usermod -aG docker $USER` *(then log out and in)*<br>`docker run hello-world` | Docker Desktop on Windows runs on WSL 2, so install WSL first. |
| **Allow PowerShell scripts** | `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser` *(PowerShell)*<br>`Get-ExecutionPolicy -List` | *(not needed)* | *(not needed)* | Fixes "running scripts is disabled on this system". RemoteSigned runs local scripts and signed downloads only. |
| **Check what is installed** | `winget list`<br>`git --version; node -v; python --version; code -v`<br>`where git` *(path to the executable)* | `brew list`<br>`git --version; node -v; python3 --version`<br>`which git` | `apt list --installed 2>/dev/null \| grep -i git`<br>`git --version; node -v; python3 --version`<br>`which git` | After installing anything, open a new terminal window. Existing shells keep the old PATH. |

## Git

| Task | Command | Notes |
|---|---|---|
| **One-time setup** | `git config --global user.name "Your Name"`<br>`git config --global user.email "you@example.com"`<br>`git config --global init.defaultBranch main`<br>`git config --list` *(show all settings)* | Settings live in ~/.gitconfig. Drop --global to set them for one repository only. |
| **Start a repository** | `git init`<br>`git clone https://github.com/user/repo.git`<br>`git clone --depth 1 URL` *(latest commit only, fast)* | |

| Task | Command | Notes |
| --- | --- | --- |
| **See what changed** | `git status`<br>`git status -s` *(short form)*<br>`git diff` *(unstaged changes)*<br>`git diff --staged` *(what will be committed)*<br>`git diff main..feature` *(between branches)* | |
| **History** | `git log --oneline --graph --all`<br>`git log -p file.txt` *(changes to one file)*<br>`git log -5 --stat`<br>`git show HEAD` *(last commit in full)*<br>`git blame file.txt` *(who changed each line)* | |
| **Stage and commit** | `git add file.txt`<br>`git add -A` *(everything)*<br>`git add -p` *(pick hunks interactively)*<br>`git commit -m "Message"`<br>`git commit -am "Message"` *(add tracked files and commit)*<br>`git commit --amend` *(fix the last commit)* | ⚠ Do not amend a commit you have already pushed; it rewrites history others may have. |
| **Unstage or discard** | `git restore --staged file.txt` *(unstage, keep changes)*<br>`git restore file.txt` *(discard local changes)*<br>`git clean -fd` *(delete untracked files and folders)*<br>`git clean -n` *(dry run first)* | ⚠ restore and clean throw work away with no undo. Run git clean -n first. |
| **Branches** | `git branch` *(list)*<br>`git branch -a` *(include remote)*<br>`git switch -c feature` *(create and switch)*<br>`git switch main`<br>`git branch -m old new` *(rename)*<br>`git branch -d feature` *(delete merged)*<br>`git branch -D feature` *(force delete)* | switch and restore replaced the overloaded checkout in Git 2.23. checkout still works everywhere. |
| **Merge and rebase** | `git merge feature` *(merge into current branch)*<br>`git rebase main` *(replay current branch onto main)*<br>`git rebase -i HEAD~3` *(squash or reorder last 3)*<br>`git merge --abort / git rebase --abort` | Merge keeps history as it happened. Rebase makes it linear. ⚠ Never rebase commits that are already pushed and shared. |
| **Remotes** | `git remote -v`<br>`git remote add origin https://github.com/user/repo.git`<br>`git remote set-url origin NEW_URL` | |

| Task | Command | Notes |
|---|---|---|
| **Push and pull** | `git push -u origin main` *(first push; sets upstream)*<br>`git push`<br>`git pull` *(fetch and merge)*<br>`git pull --rebase`<br>`git fetch` *(download without merging)*<br>`git push origin --delete feature` *(delete remote branch)* | |
| **Undo commits** | `git revert HEAD` *(new commit that undoes the last one; safe)*<br>`git reset --soft HEAD~1` *(uncommit, keep changes staged)*<br>`git reset --mixed HEAD~1` *(uncommit, keep changes unstaged)*<br>`git reset --hard HEAD~1` *(uncommit and discard changes)*<br>`git reflog` *(find lost commits)* | ⚠ reset --hard deletes work. revert is the safe choice for anything already pushed. reflog can recover a reset for about 30 days. |
| **Stash** | `git stash` *(shelve changes)*<br>`git stash push -m "wip"` *(with a name)*<br>`git stash list`<br>`git stash pop` *(restore latest and drop it)*<br>`git stash apply stash@{1}`<br>`git stash drop` | |
| **Tags and releases** | `git tag` *(list)*<br>`git tag -a v1.0 -m "Release 1.0"`<br>`git push --tags`<br>`git checkout v1.0` *(look at a tag)* | |
| **Ignore and untrack files** | `git rm --cached file.txt` *(stop tracking, keep file)*<br>`git rm -r --cached folder/`<br>`git check-ignore -v file.txt` *(why is this ignored?)*<br>`git mv old.txt new.txt` *(rename and stage)* | Patterns in .gitignore only affect files not yet tracked. Untrack committed files with git rm --cached first. |
| **Search** | `git grep "pattern"` *(search tracked files)*<br>`git log -S "text"` *(commits that added or removed text)*<br>`git log --grep "fix"` *(search commit messages)*<br>`git log --author="Name"` | |
| **Who did what** | `git shortlog -sn` *(commits per author)*<br>`git log --since="2 weeks ago"`<br>`git bisect start / git bisect good v1.0 / git bisect bad` *(find the breaking commit)* | |
| **Help** | `git help commit`<br>`git commit -h` *(short flag list)*<br>`git <command> --help` | |

# Docker

| Task | Command | Notes |
|---|---|---|
| **List containers** | `docker ps` *(running)*<br>`docker ps -a` *(include stopped)*<br>`docker ps -q` *(IDs only)* | On Linux, prefix with sudo or add yourself to the docker group. Docker Desktop on Windows and macOS runs the daemon for you. |
| **List images** | `docker images`<br>`docker image ls`<br>`docker history nginx` *(layers)* | |
| **Run a container** | `docker run -d -p 8080:80 --name web nginx` *(detached, port map, named)*<br>`docker run -it ubuntu bash` *(interactive shell)*<br>`docker run --rm alpine echo hi` *(remove when it exits)*<br>`docker run -v ${PWD}:/app node` *(mount current folder)* | -d runs in the background, -p maps host:container ports, -v mounts a volume, --rm cleans up on exit. |
| **Shell into a container** | `docker exec -it web bash`<br>`docker exec -it web sh` *(if bash is missing)*<br>`docker attach web` *(attach to PID 1; Ctrl+P Ctrl+Q to detach)* | |
| **Logs** | `docker logs web`<br>`docker logs -f web` *(follow)*<br>`docker logs --tail 50 web` | |
| **Stop, start, remove** | `docker stop web`<br>`docker start web`<br>`docker restart web`<br>`docker rm web` *(add -f to force)*<br>`docker rm -f $(docker ps -aq)` *(remove all ⚠)* | ⚠ The last command removes every container. |
| **Remove images** | `docker rmi nginx`<br>`docker image rm nginx:latest`<br>`docker image prune` *(dangling)*<br>`docker image prune -a` *(all unused)* | |
| **Free up space** | `docker system df` *(show usage)*<br>`docker system prune` *(stopped containers, unused networks, dangling images)*<br>`docker system prune -a --volumes` *(⚠ everything unused, including volumes)* | ⚠ prune -a --volumes deletes data in unused volumes. Check docker volume ls first. |
| **Build an image** | `docker build -t myapp .`<br>`docker build -t myapp:1.0 -f Dockerfile.prod .`<br>`docker tag myapp myrepo/myapp:1.0` | The trailing dot is the build context, the folder sent to the daemon. |

| Task | Command | Notes |
|---|---|---|
| **Push and pull** | `docker pull redis:7`<br>`docker login`<br>`docker push myrepo/myapp:1.0` | |
| **Copy files** | `docker cp web:/etc/nginx/nginx.conf ./nginx.conf` *(out of the container)*<br>`docker cp ./file.txt web:/tmp/` *(into the container)* | |
| **Inspect and stats** | `docker inspect web` *(full JSON)*<br>`docker stats` *(live CPU and memory)*<br>`docker port web`<br>`docker top web` *(processes)* | |
| **Volumes and networks** | `docker volume ls`<br>`docker volume create data`<br>`docker network ls`<br>`docker network create mynet` | |
| **Docker Compose** | `docker compose up -d` *(start in background)*<br>`docker compose down` *(stop and remove)*<br>`docker compose ps`<br>`docker compose logs -f`<br>`docker compose build`<br>`docker compose exec web sh` | docker compose (v2, a subcommand) replaces the old docker-compose script. It reads compose.yaml or docker-compose.yml in the current folder. |

## Building PowerShell One-Liners

Windows gives you two shells that look almost identical but obey different rules: **Command Prompt** (`cmd.exe`) and **Windows PowerShell** 5.1. This section is about reading a one-liner, understanding each piece, and assembling your own, rather than copying blindly. The worked examples at the end are real commands broken down part by part.

### 1. Which shell am I in?

Look at the prompt. `PS C:\Users\you>` is PowerShell; `C:\Users\you>` with no `PS` in front is Command Prompt. They share many names (`cd`, `dir`, `cls`) but the syntax underneath differs, so a line that works in one can fail in the other.

- Open PowerShell: Start menu, type **PowerShell**. Open Command Prompt: type **cmd**.
- Open either as administrator: right-click it and choose **Run as administrator**, or press Ctrl+Shift+Enter when launching it from the Start menu. The title bar then reads "Administrator".
- Hop between them: type `powershell` to enter PowerShell, or `cmd` to drop into Command Prompt. Type `exit` to go back.
- Run one Command Prompt command from PowerShell with `cmd /c "..."` (used in example E).

Rule of thumb: if a line uses `$variables`, pipes `|` into a cmdlet, or has `{ }` blocks, it is PowerShell. If it uses slash flags like `/S /Q`, it is Command Prompt style.

### 2. PowerShell building blocks

- `$name = value` stores a value. `$log = 'C:\...\out.txt'` holds a path you reuse later in the same line.
- `|` (the pipeline) sends each object from the command on the left to the command on the right. See **Redirection and Piping** for the table.
- `ForEach-Object { ... }` runs the block once per piped object; inside it `$_` is the current object, and `$_.Name` reads that object's Name property.
- `foreach ($x in $list) { ... }` is the loop form for when you already have a list, rather than a pipeline (example B).
- `;` runs several commands in order on one line, left to right. See **Chain commands** under Environment, History and Shell Basics.
- `{ ... }` is a script block: code passed around as a single unit, such as the body of ForEach-Object.
- `| Out-Null` throws output away when you only want the side effect. See **Suppress output**.

### 3. Quoting, and the $ trap

- Single quotes `'...'` are literal: nothing inside is interpreted.
- Double quotes `"..."` expand variables. `"$($_.Name)"` runs the expression `$($_.Name)` and drops the result into the string; the `$( )` tells PowerShell to evaluate the property instead of printing the whole object.
- ⚠ A `$` in a path must be inside single quotes. In double quotes or unquoted, PowerShell treats `$RECYCLE` as a variable, which is empty.

That last point bites in a real way. Unquoted, this path collapses:

```
E:\$RECYCLE.BIN\   ->   E:\.BIN\     (wrong: $RECYCLE read as an empty variable)
'E:\$RECYCLE.BIN'                       (right: single quotes keep the $ literal)
```

### 4. Aliases that lie: Command Prompt habits that break in PowerShell

PowerShell defines `dir`, `del` and `rmdir` as aliases, but they point to cmdlets with different switches, so the Command Prompt flags fail.

- `rmdir /S /Q folder` works in Command Prompt. In PowerShell `rmdir` is `Remove-Item`, which has no `/S /Q` and errors. The PowerShell form is `Remove-Item folder -Recurse -Force`.
- `dir` is `Get-ChildItem`; the Command Prompt hidden-files switch `dir /A` is `Get-ChildItem -Force`.
- `del` is `Remove-Item`; `del /Q` does nothing useful, use `-Force`.
- To deliberately use the Command Prompt version from PowerShell, wrap it: `cmd /c "rmdir /s /q folder"`.

The full alias list is in **Notes and Gotchas**, under "PowerShell is different".

### 5. Robocopy as a read-only inspector

Robocopy is a copy tool, but with `/L` it copies nothing and only reports what it would do. That makes it a fast way to list or size folders, including protected ones a normal account cannot read. ⚠ Without `/L` it really copies, so always include `/L` when you are only inspecting.

- `/L` list only: report, do not copy. The safety flag.

- `/E` include all subfolders, even empty ones.
- `/B` backup mode: read files the normal account cannot. 🔒 Needs administrator.
- `/BYTES` show sizes in bytes instead of rounded units.
- `/TS` add timestamps and `/FP` full paths to the listing.
- `/NC` no class, `/NDL` no directory list, `/NJH` no job header, `/NJS` no job summary, `/NFL` no file list: trim the output down to the lines you want.
- `/R:0` do not retry failed files and `/W:0` wait zero seconds: fail fast rather than hang on locked files.
- `/UNILOG:file` write a Unicode log to that file (paired with `$log` in example A).

Robocopy's exit codes are unusual: 0 means nothing to do, 1 means files were (or would be) copied, and anything below 8 is success. 8 or higher is a real error. So a non-zero exit from robocopy is normal, not a failure.

### 6. Filtering and shaping output

These cmdlets chain after a `|` to cut a firehose down to what you want. Each already has a reference row; here is when to reach for which.

- `Select-String 'pattern'` keeps only matching lines, like grep. Example B uses `Select-String 'Files :|Bytes :'` to keep just the summary lines. See **Search text in files**.
- `Where-Object { $_.X -gt 1 }` keeps objects that match a condition. See **Filter a stream**.
- `Select-Object A,B` picks columns; `Select-Object -First 10` takes the first rows.
- `Sort-Object Size -Descending` orders objects.
- `Format-Table -AutoSize` lays the result out as aligned columns (example C). Put it last; it is for display only. See **Shape and export output**.

### 7. Reading system and disk info

- `Get-PhysicalDisk` lists the physical drives; pipe each into `Get-StorageReliabilityCounter` for temperature, read and write error counts, and power-on hours (example A).
- `Get-CimInstance Win32_LogicalDisk` lists volumes with size and free space (example C). Also in **Disk and Storage**.
- `vssadmin list shadowstorage` shows the space used by Volume Shadow Copies and restore points. 🔒 Needs administrator.
- `icacls path` shows or changes the Windows permissions on a file or folder. See **File permissions**.

### 8. Safety habits

- **Look before you delete.** List the target first with `robocopy ... /L`, `dir`, or `Get-ChildItem`, and read it.
- **Preview, then apply.** Good scripts take a switch so the plain run only reports and a second run acts (example D).
- **Know what needs admin.** `/B`, `vssadmin`, and changing permissions or services all need an Administrator window (🔒).
- ⚠ **Some actions cannot be undone.** `Remove-Item -Recurse -Force`, `rd /s /q`, `del`, and formatting all bypass the Recycle Bin. There is no undo. Example E is shown to read, not to run.

### Worked example: build example A step by step

Example A saves a file listing, then prints drive health. Grow it in stages.

Start by listing a folder without copying anything:

```
robocopy 'E:\found.000' 'C:\robocopy_list_only' /L /E
```

Trim the output, fail fast on locked files, and send it to a log held in a variable:

```
$log = 'C:\Users\randu\AppData\Local\Temp\found_E_listing.txt'
robocopy 'E:\found.000' 'C:\robocopy_list_only' /L /E /B /BYTES /TS /FP /NC /NDL /NJH /NJS /R:0 /W:0 /UNILOG:$log | Out-Null
```

The `| Out-Null` hides robocopy's console chatter because you only want the log file. Now add the drive-health read after a `;`:

```
Get-PhysicalDisk | ForEach-Object { $c = $_ | Get-StorageReliabilityCounter; "$($_.DeviceId) $($_.FriendlyName)
temp=$($c.Temperature) readErrors=$($c.ReadErrorsUncorrected) writeErrors=$($c.WriteErrorsUncorrected)
hours=$($c.PowerOnHours)" }
```

For each physical disk, `$c` holds its reliability counters, and the double-quoted string pulls named fields out with `$( )`. Join the two halves with `;` and end with `'done'`, and that is the whole of example A.

### More worked examples

**B. Size several protected folders.** A `foreach` loop runs robocopy on each path and keeps only the summary lines:

```
foreach ($p in 'E:\found.000','E:\System Volume Information','G:\found.000') { $p; robocopy $p 'C:\robocopy_list_only' /L /E
/B /BYTES /NFL /NDL /NJH /R:0 /W:0 | Select-String 'Files :|Bytes :' }; vssadmin list shadowstorage /for=E:
```

- `foreach ($p in a,b,c)` loops over the three paths; `$p;` prints the path as a header before its totals.
- `/NFL` (no file list) joins the trims from section 5, leaving just the totals; `Select-String 'Files :|Bytes :'` keeps those two lines.
- `/B` needs admin. `vssadmin list shadowstorage /for=E:` at the end reports shadow-copy usage on E:.

**C. List drive sizes and free space as a clean table:**

```
Get-CimInstance Win32_LogicalDisk | Select-Object DeviceID, VolumeName, @{n='SizeGB';e={[math]::Round($_.Size/1GB,1)}},
@{n='FreeGB';e={[math]::Round($_.FreeSpace/1GB,1)}} | Format-Table -AutoSize
```

- `@{n='SizeGB';e={...}}` is a calculated column: `n` is its name, `e` is the expression. Here it turns bytes into GB with `[math]::Round(.../1GB,1)`.
- `Format-Table -AutoSize` aligns it. Change `1GB` to `1MB`, or the rounding `1` to `2`, to reshape it.

**D. Run a script preview-then-apply.** Two runs of the same script:

```
powershell -ExecutionPolicy Bypass -File C:\path\to\script.ps1
powershell -ExecutionPolicy Bypass -File C:\path\to\script.ps1 -Apply
```

- `-ExecutionPolicy Bypass` lets this one script run without permanently changing your machine's policy. See **Allow PowerShell scripts** in Notes.
- The first line has no `-Apply`, so a well-written script only previews. The second adds `-Apply` to make the changes. Preview, read it, then apply.

**E. The same delete in each shell, shown for explanation only:**

```
Command Prompt:  rd /s /q "E:\$RECYCLE.BIN"
PowerShell:      cmd /c 'rd /s /q "E:\$RECYCLE.BIN"'
```

- `rd /s /q` is Command Prompt: `/s` recurse, `/q` quiet. It permanently deletes the folder and everything under it.
- In PowerShell, `rd` would be `Remove-Item` and would mis-read `$RECYCLE`, so the line wraps the real Command Prompt call in `cmd /c '...'`, using single quotes to protect the `$`.
- ⚠ This is permanent and cannot be undone. It is here to read, not to run.

## Keyboard Shortcuts

| Action | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Auto-complete** | `Tab` | `Tab` *(Ctrl+Space shows a menu)* | `Tab` *(press twice to list options)* | |
| **Previous / next command** | `↑ / ↓` *(F3 repeats the last one)* | `↑ / ↓` | `↑ / ↓` or `Ctrl+P / Ctrl+N` | |
| **Search history** | `F7 (list), F8` *(cycle prefix matches)* | `Ctrl+R (backward), Ctrl+S (forward), F8` | `Ctrl+R (backward), Ctrl+S` *(forward)* | |
| **Clear screen** | `cls` *(no shortcut)* | `Ctrl+L` | `Ctrl+L` | |
| **Cancel running command** | `Ctrl+C` | `Ctrl+C` | `Ctrl+C` | |
| **Clear current line** | `Esc` | `Esc` | `Ctrl+U (before cursor), Ctrl+K` *(after cursor)* | In PowerShell, Ctrl+U and Ctrl+K also work with the default key mode. |
| **Start / end of line** | `Home / End` | `Home / End` *(also Ctrl+A / Ctrl+E)* | `Ctrl+A / Ctrl+E` *(Home / End in most terminals)* | |
| **Move by word** | `Ctrl+← / Ctrl+→` | `Ctrl+← / Ctrl+→` | `Alt+B / Alt+F` *(Ctrl+← / → in many terminals)* | |

| Action | CMD | PowerShell | Bash | Notes |
|---|---|---|---|---|
| **Delete word before cursor** | *(none)* | `Ctrl+Backspace` | `Ctrl+W` | |
| **Delete word after cursor** | *(none)* | `Ctrl+Delete` | `Alt+D` | |
| **Undo an edit** | *(none)* | `Ctrl+Z` | `Ctrl+_ or Ctrl+X Ctrl+U` | |
| **Paste** | `Ctrl+V or right-click` | `Ctrl+V or right-click` | `Ctrl+Shift+V` *(Cmd+V on macOS)* | Copy and paste are handled by the terminal app, not the shell. In Bash Ctrl+C cancels and Ctrl+V inserts a literal key. |
| **Copy** | `Select, then Enter or Ctrl+C` | `Select, then Enter or Ctrl+C` | `Ctrl+Shift+C` *(Cmd+C on macOS)* | |
| **Pause / resume output** | `Ctrl+S / Ctrl+Q` | `Ctrl+S / Ctrl+Q` | `Ctrl+S / Ctrl+Q` | If the terminal seems frozen, you probably hit Ctrl+S. Press Ctrl+Q. |
| **Suspend foreground job** | *(none)* | *(none)* | `Ctrl+Z` *(resume with fg or bg)* | |
| **Exit shell** | `exit` | `exit` *(Ctrl+D on an empty line)* | `exit or Ctrl+D` | In CMD, Ctrl+Z is the end-of-file character, not exit. |

## Notes and Gotchas

### Paths

- Windows uses backslashes (C:\Users\Alice). Bash uses forward slashes (/home/alice). PowerShell accepts both.
- Quote any path containing spaces: "C:\Program Files" or '/My Files'.
- Windows file systems are case-insensitive. Linux is case-sensitive: File.txt and file.txt are different files. macOS is case-insensitive by default.

### Quoting and escaping

- CMD: escape special characters (& < > | ^) with a caret: echo a ^& b. Variables are %NAME%.
- PowerShell: single quotes are literal, double quotes expand $variables and $(expressions). The escape character is the backtick (`).
- Bash: single quotes are literal, double quotes expand $variables and $(commands). The escape character is the backslash (\).

### Wildcards

- * matches any number of characters (*.txt). ? matches exactly one character (file?.txt).
- [abc] and [a-z] match one character from a set (Bash and PowerShell, not CMD).
- In Bash the shell expands wildcards before the command runs. Quote them ("*.txt") when the command should see the pattern itself, as with find and grep.

### PowerShell is different

- Commands are Verb-Noun cmdlets (Get-Process, Stop-Service). Aliases such as ls, cat, rm and ps map to cmdlets, but their flags are PowerShell flags: ls -Force, not ls -a.
- Pipelines carry objects, not text. Filter with Where-Object, pick columns with Select-Object, sort with Sort-Object, and format last.
- Windows PowerShell 5.1 ships with Windows. PowerShell 7 (pwsh) is a separate install with && and ||, UTF-8 defaults and Get-Uptime.

### macOS, zsh and WSL

- macOS uses zsh as its default shell. Nearly everything in the Bash column works unchanged; profile files are ~/.zshrc instead of ~/.bashrc.
- macOS ships BSD versions of many tools, so some Linux flags differ (sed -i, ls --color, du --max-depth). Commands marked (Linux) do not exist on macOS and vice versa.
- Windows Subsystem for Linux (wsl) gives you a real Linux shell on Windows. Windows drives appear under /mnt/c.

### Setting up Windows for development

- Turn on Developer Mode (Settings > System > For developers). It lets you create symlinks without an admin prompt and, on Windows 11 24H2+, enables the built-in sudo.
- Long paths: node_modules trees often exceed the 260-character limit. Run git config --global core.longpaths true, and enable "Enable Win32 long paths" in Group Policy or set LongPathsEnabled=1 under HKLM\SYSTEM\CurrentControlSet\Control\FileSystem (🔒 admin).
- winget installs per user by default; add --scope machine (🔒 admin) to install for everyone.
- Most installers add themselves to the PATH, but only new terminal windows see it. If a command is "not recognized" right after installing, open a new window first.

### Git on Windows

- Git for Windows installs Git Bash, a Bash shell where the whole Bash column of this sheet works. Git itself works identically in CMD and PowerShell.
- Line endings: set git config --global core.autocrlf true on Windows (or add a .gitattributes with * text=auto to the repo) so files do not flip between CRLF and LF.
- Credentials: Git for Windows includes Git Credential Manager, which opens a browser sign-in on the first push and remembers it. On macOS the keychain does the same; on Linux use git config --global credential.helper store or the GitHub CLI (gh auth login).
- Lots of "modified" files right after a clone with no real changes usually means a line-ending or file-mode mismatch: git config core.fileMode false and check core.autocrlf.

Command Line Interface Cheat Sheet · © 2026 Fulllion Creative Works · MIT License · also available as DOCX and PDF in the repository.